

On the Design of Hard mUBQP Instances

Murilo Zangari
Federal University of Parana,
Curitiba, Brazil
murilo.zangari@gmail.com

Roberto Santana
University of the Basque
Country (UPV/EHU), San
Sebastián, Spain
roberto.santana@ehu.es

Alexander Mendiburu
University of the Basque
Country (UPV/EHU), San
Sebastián, Spain
alexander.mendiburu@ehu.es

Aurora Pozo
Federal University of Parana,
Curitiba, Brazil
aurora@inf.ufpr.br

ABSTRACT

This paper proposes a new method for the design and analysis of multi-objective unconstrained binary quadratic programming (mUBQP) instances, commonly used for testing discrete multi-objective evolutionary algorithms (MOEAs). These instances are usually generated considering the sparsity of the matrices and the correlation between objectives but randomly selecting the values for the matrix cells. Our hypothesis is that going beyond the use of objective correlations by considering different types of variables interactions in the generation of the instances can help to obtain more diverse problem benchmarks, comprising harder instances. We propose a parametric approach in which small building blocks of deceptive functions are planted into the matrices that define the mUBQP. The algorithm for creating the new instances is presented, and the difficulty of the functions is tested using variants of a decomposition-based MOEA. Our experimental results confirm that the instances generated by planting deceptive blocks require more function evaluations to be solved than instances generated using other methods.

CCS Concepts

•Mathematics of computing → Discrete mathematics; Combinatorial optimization; •Computing methodologies → Artificial intelligence; Discrete space search;

1. INTRODUCTION

The design of function benchmarks able to capture different facets of difficulty of multi-objective problems (MOPs) is one of the avenues to understand the difficulties that these problems represent for multi-objective evolutionary algorithms (MOEAs). A number of functions test sets have been proposed in the literature [5,8] and they continue to be

extensively applied for benchmarking MOEAs. These benchmarks are mainly focused on continuous problems. However, research on combinatorial MOPs can provide insights about how the structural characteristics of the objective functions influence the shape and other attributes of the Pareto fronts [1, 21, 24, 25].

In this paper, we introduce a new method for generating difficult instances for a multi-objective variant of the unconstrained binary quadratic programming (UBQP) [10]. UBQP is a flexible model with a very general formulation. The multiobjective UBQP (mUBQP) was proposed in [11]. In the same paper, a method for generating instances of the problem was proposed. The idea was to guarantee, by construction, that instances will satisfy a number of characteristics related to the sparsity of the single-objective problems and the degree of correlation between the objectives. Both characteristics have an important impact in the behavior of MOEAs. In this paper, we follow a different path to the construction of mUBQP instances. Our proposal is based on planting deceptive modules in the matrices describing the structure of the objective functions. We build on previous results on the analysis of deceptive MOPs [18] and on measures to describe levels of difficulty for MOPs.

The paper is organized as follows: In the next section, some background on the mUBQP problem is provided. Section 3 discusses research related to our proposal. Our approach to construct instances of the mUBQP is explained in Section 4. Section 5 explains how functions of varying difficulty are generated. In Section 6, the decomposition-based approach to the mUBQP problem is presented. Section 7 describes the experimental framework used and presents the numerical results. The main contributions of the paper are summarized in Section 8, where some lines for future research are also discussed.

2. PRELIMINARIES

Let $\mathbf{X} = (X_1, \dots, X_n)$ denote a vector of discrete random variables. We use $\mathbf{x} = (x_1, \dots, x_n)$ to denote an assignment to the variables. We will represent a population as a set of vectors $\mathbf{x}^1, \dots, \mathbf{x}^N$ where N is the size of the population. Similarly, x_i^l will represent the assignment to the i th variable of the l th solution in the population.

A general MOP can be defined as follows [3]:

$$\text{minimize } F(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x})); \quad \text{subject to } \mathbf{x} \in \Omega \quad (1)$$

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GECCO '16, July 20-24, 2016, Denver, CO, USA

© 2016 ACM. ISBN 978-1-4503-4206-3/16/07...\$15.00

DOI: <http://dx.doi.org/10.1145/2908812.2908889>

where $\mathbf{x} = (x_1, x_2, x_n)^T$ is the decision variable vector of size n , Ω is the *decision space*, $F : \Omega \rightarrow R^m$ consists of m objective functions and R^m is the *objective space*.

Let $\mathbf{x}, \mathbf{y} \in \Omega$, $\mathbf{x}$ is said to *dominate* $\mathbf{y}$ if and only if $f_i(\mathbf{x}) \leq f_i(\mathbf{y})$ for all $i \in \{1, \dots, m\}$ and $f_i(\mathbf{x}) < f_i(\mathbf{y})$ for at least one i . A solution $\mathbf{x}^* \in \Omega$ is called *Pareto optimal* if there is no other $\mathbf{x} \in \Omega$ which *dominates* $\mathbf{x}^*$. The set of all the Pareto optimal solutions is called the *Pareto set (PS)* and the solutions mapped in the *objective space* are called *Pareto front (PF)*, i.e., $PF = \{F(\mathbf{x}) | \mathbf{x} \in PS\}$.

In the mUBQP, there exists a collection of n items, and each pair of items is associated to $m \geq 2$ profit values that can be positive, negative or zero. Formally, for each objective k , a symmetric $n \times n$ matrix $Q^k = (q_{ij}^k)$ is given, and a binary vector $\mathbf{x} = (x_1, \dots, x_n)$, $x_i \in \{0, 1\}$ is searched, such that the m objective functions are maximized [11]:

$$\max F(\mathbf{x}) = \sum_{i=1}^n \sum_{j=1}^n q_{ij}^k x_i x_j; \quad k \in \{1, \dots, m\}; \quad (2)$$

where $F(\mathbf{x}) = (f_1(\mathbf{x}), \dots, f_m(\mathbf{x}))$ is an objective function vector.

3. RELATED WORK

A number of papers have proposed benchmarks [5, 8] for evaluating different facets of the difficulty of MOPs. These benchmarks mainly comprise continuous functions and emphasis is put on the shape of the Pareto front and not on the relationships between the decision variables.

Knowles and Corne [9] introduced an instance generator for the multiobjective quadratic assignment problem (mQAP) and investigated the effect of correlations between the objectives. To enforce the correlations between the mQAP objectives, a correlation parameter c is used. Based on this fixed parameter, the instances are generated.

Aguirre et al. [1] present an exhaustive investigation of how the parameters of the MNK-landscape influence several characteristics of the fitness landscape, including the size of the Pareto front and the number of fronts.

Verel et al. [25] extended the NK-model to design multiobjective functions with correlations. In the ρ MNK-landscapes, the epistatic structure is the same for all the objectives, and the local fitness functions are not independent. In [24], a complete enumeration of the small ρ MNK-landscape instances was used to investigate the effect of the correlations in the characteristics of the Pareto front. In a recent work, the ρ MNK-landscape model has been used to investigate the influence of the parameters describing the landscapes in the behavior of different MOEAs [6, 14], showing the general usefulness of this type of models. The approach followed to create instances is different to the one presented here.

Much work in evolutionary computation has been devoted to study deception for discrete problems. The proposal presented in this paper is inspired by previous analysis of multiobjective deceptive functions [4, 15, 18].

4. PLANTING PATCHES OF DECEPTIVE SUBFUNCTIONS IN MUBQP MATRICES

In this section, we propose a bottom-up approach in which the mUBQP matrices are filled by the addition of small deceptive submatrices.

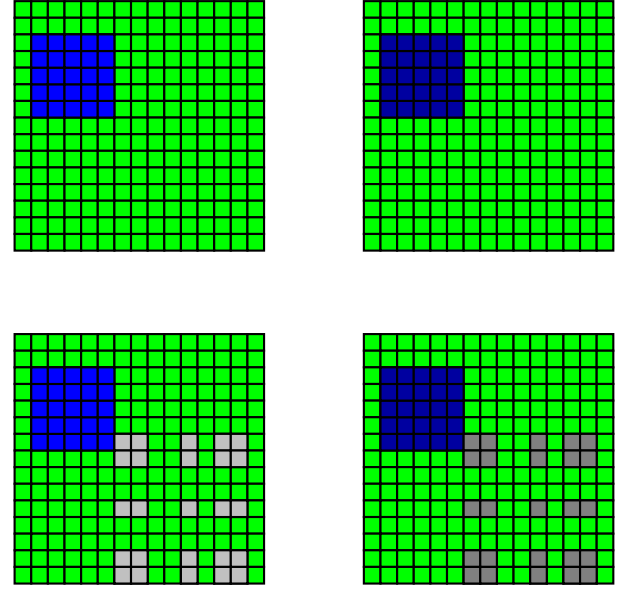


Figure 1: Two steps in the construction of an instance of a bi-objective UBQP problem ($n = 15$). Top row: two compact submatrices ($n_p = 5$) containing the relationships between variables $\{X_2, X_3, X_4, X_5, X_6\}$ are planted. Bottom row: two submatrices ($n_p = 5$) are planted containing the relationships between variables $\{X_7, X_8, X_{11}, X_{13}, X_{14}\}$.

The algorithm starts from m sets of zero matrices and, in each step, a single submatrix is added to each of the m matrices. Each submatrix is “planted” in the same position for the m mUBQP matrices. The process is illustrated in Figure 1, where we show an example of a bi-objective problem of size $n = 15$. In the figure, green cells correspond to matrix entries equal to zero. In the first step (top matrices), there is one submatrix ($n_p = 5$) added to each matrix. As it can be appreciated in the figure, they are planted in the same position in the two matrices. To emphasize the fact that the planted submatrices are related, we represent them using similar but not identical colors. In the second step (bottom matrices), another pair of submatrices is added. Notice that submatrices do not need to contain contiguous variables. For instance, the second set of submatrices shown in Figure 4 contains variables $\{X_7, X_8, X_{11}, X_{13}, X_{14}\}$.

There are a number of key issues relevant to our approach: 1) How are the submatrices created? 2) How to select the position to insert the submatrices? 3) How many submatrices to add? In the following sections, we discuss these issues.

4.1 Parametrization of the UBQP problem

To create a submatrix, we parameterize a small (single-objective) UBQP problem of $n_p = 5$ variables in terms of the interactions between its variables q_{ij} . Equation (3) shows the expression for the general parameterization of such UBQP problem.

$$Q = \begin{pmatrix} 0 & q_{12} & q_{13} & q_{14} & q_{15} \\ q_{12} & 0 & q_{23} & q_{24} & q_{25} \\ q_{13} & q_{23} & 0 & q_{34} & q_{35} \\ q_{14} & q_{24} & q_{34} & 0 & q_{45} \\ q_{15} & q_{25} & q_{35} & q_{45} & 0 \end{pmatrix} \quad (3)$$

We constrain the possible values each cell can take and set $q_{ij} \in \{-1, 1\}$ and $q_{ii} = 0$. For $n_p = 5$, there are $\frac{n_p(n_p-1)}{2} = 10$ pairwise interactions contained in this submatrix. Considering the two values each parameter can take, the number of possible UBQP functions that can be generated is $|\mathcal{F}| = 2^{10} = 1024$, where $\mathcal{F}$ is the set of functions, and each function corresponds to a different submatrix.

It is important to emphasize, that we refer here to the space of UBQP matrices that satisfy the constraints of symmetry ($q_{ij} = q_{ji}$) for a very reduced number of values of interactions ($q_{ij} \in \{-1, 1\}$). Each of the 1024 matrices defines a different problem of size $n = 5$ and therefore, for each of such problems we could find an optimal binary solution $\mathbf{x} = (x_1, x_2, x_3, x_4, x_5)$ that maximizes the function for this particular choice of matrix Q .

4.2 Deceptive UBQP functions

Average-sense misleading (deceptive) functions [7] have played a fundamental role in the analysis of GAs. In deceptive functions, there are at least two optima, a true optimum and a deceptive optimum. Commonly applied search operators tend to favor the deceptive optimum making very difficult for the algorithm to find the true optimum.

$$\text{trap}_5(u) = \begin{cases} 5 & \text{if } u = 5 \\ 4 - u & \text{if } u < 5 \end{cases} \quad (4)$$

$$\text{invtrap}_5(u) = \begin{cases} 5 & \text{if } u = 0 \\ u - 1 & \text{if } u > 0 \end{cases} \quad (5)$$

Pelikan et al. [18] used a simple bi-objective deceptive function for analyzing the behavior of EAs. The bi-objective trap problem comprises unitation functions trap_5 (Equation (4)) and invtrap_5 (Equation (5)) where $u(\mathbf{x}) = \sum_i x_i$. These functions are conflicting, and since the global optimum is reached in a single point, for a problem of $n = 5l$ variables, the number of solutions in the Pareto set is 2^l [18]. This bi-objective problem can deceive MOEAs that do not consider interactions between the variables away from the optimum.

We use the bi-trap function as inspiration to propose a way to construct deceptive pairs of UBQP submatrices. We start from all possible functions obtained by the parameterization proposed in Section 4.1. From the space of $|\mathcal{F}| = 2^{10} = 1024$ functions, we would like to identify which ones are deceptive. First, we need a more formal definition of deception [26].

Let h denote an $(n - j)$ -dimensional hyperplane where j variables have preassigned bit values and $\alpha(h)$ be a mask with 1 bits marking the locations where the j variables appear in the problem location and 0 elsewhere. Let $\text{MAX}(\mathbf{x}, \alpha(h))$ return the hyperplane with the best mean over all 2^j order j hyperplanes that can be defined using the $\alpha(h)$ mask.

A function is *order- j deceptive* [26] if the j bit values returned by $\text{MAX}(\mathbf{x}, \alpha(h))$ for all hyperplanes of order- j are not the same as the bit values found in a string which is a global optimum.

We use, a more relaxed definition of an order- j deceptive function, in which a function is *mildly order-1 deceptive* if the j bit values returned by $\text{MAX}(\mathbf{x}, \alpha(h))$ for at least one hyperplane of order j is not the same as the bit values found in a string which is a global optimum.

In the particular case of a mildly *order-1* deceptive function, the definition requires that the function will be deceptive if for at least one of the n order-1 hyperplanes the value of $\text{MAX}(\mathbf{x}, \alpha(h))$ will be different to the optimum. For small values of n it is very easy to check if a function is mildly deceptive. We can evaluate the entire space of solutions, compute the order-1 hyperplanes, and check whether there is any hyperplane $\text{MAX}(\mathbf{x}, \alpha(h))$ that is not in any global optimum. However, for the construction of the mUBQP instances we are not just interested in determining if a function is deceptive but also in measuring the degree of deception. We propose a way to do this based on the computation of univariate probabilities.

4.3 Computing the degree of deception

One easy way to test if a function is deceptive from a sufficiently large population of solutions distributed according to the fitness is by computing the univariate frequencies of the variables. If the most frequent configurations for the single variables are not consistent with the configurations of these variables in the global optimum, then the function is mildly order-1 deceptive.

For a given matrix Q , we compute the fitness for all the $2^5 = 32$ configurations of $\mathbf{x} = (x_1, x_2, x_3, x_4, x_5)$. Using the UBQP fitness values as an energy function, we associate to each possible solution a probability value $p(\mathbf{x})$ according to the Boltzmann probability distribution. The Boltzmann probability distribution $p_B(\mathbf{x})$ is defined as

$$p_B(\mathbf{x}) = \frac{e^{\frac{g(\mathbf{x})}{T}}}{\sum_{\mathbf{x}'} e^{\frac{g(\mathbf{x}')}{T}}}, \quad (6)$$

where $g(\mathbf{x})$ is a given objective function and T is the system temperature that can be used as a parameter to simulate the effect of the strength of selection in the frequency of the solutions. In this paper, we use $T = 1$. The Boltzmann or Gibbs distribution has the convenient property of assigning an exponentially higher probability to solutions based on their fitness. It has been used before to study the influence of variables interactions in the behavior of EAs in single-objective problems [17, 22] and MOPs [20]. By choosing $T = 1$ we set the probabilities to depend only on the fitness function.

Notice that, for small UBQP instances as the ones we use for planting solutions, it is easy to compute the Boltzmann distribution since the size of the search space is small (32). Using the Boltzmann distribution, we can also compute the univariate probabilities for each variable (e.g. $p_1(x_1 = 1) = \sum_{x', x'_1=1} p_B(\mathbf{x}')$). Then, it is possible to compute the probability of the optimum given by the univariate factorization $p_u(\mathbf{x}) = \prod_i p_B(x_i)$.

We use the univariate factorization to rank all solutions from the one with the highest probability to the one with the smallest probability. If the univariate factorization gives to the optimal solution the highest probability, then the degree of deception is minimum (1). However, as the rank given by univariate factorization increases it means that the function is more deceptive. Since for $n = 5$ there are $2^5 = 32$ possible solutions, the highest value of deception is 32. Algorithm 1 summarizes the steps to compute the degree of deception for a function determined by a matrix Q .

We computed the degree of deception for each of the 1024

Algorithm 1: Degree of deception for function $f(\mathbf{x})$

- 1 Compute the Boltzmann distribution $p_B(\mathbf{x})$ of function $f(\mathbf{x})$ on the 2^5 solutions.
 - 2 Compute the univariate probabilities from p_B .
 - 3 For each solution, compute its univariate factorization $p_u(\mathbf{x}) = \prod_i p_B(x_i)$
 - 4 Rank the solutions from the highest to lowest $p_u(\mathbf{x})$
 - 5 The average rank of the optimal solutions is the degree of deception
-

functions in $\mathcal{F}$. This information is shown in Figure 2 where functions are sorted in lexicographical order.

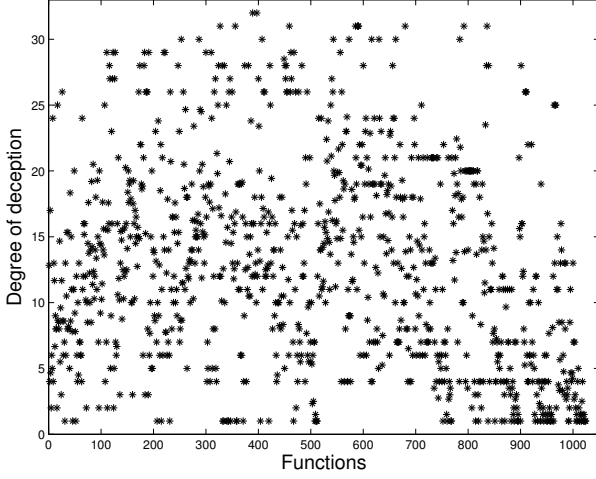


Figure 2: Degree of deception for all possible UBQP functions, $n = 5$, $q_{ij} = \{-1, 1\}$.

4.4 Deceptive mUBQP functions

To analyze the multi-objective case, we then consider all possible bi-objective functions such that $g_a, g_b \in \mathcal{F}$ and $g_a \neq g_b$. We call this set $\mathcal{O}$, where $|\mathcal{O}| = \frac{1024 \times 1023}{2} = 523776$. The idea is to identify which are the functions in $\mathcal{O}$ that satisfy conditions similar to the multi-objective trap functions, i.e., they are deceptive for the two single objective problems involved, and the Pareto set is as large as possible.

We computed the Pareto sets of all the bi-objective functions in $\mathcal{O}$. We found that the maximum size of the Pareto fronts for functions in $\mathcal{O}$ is 8, and the maximum number of Pareto optimal solutions is 32. Table 1 shows the number of generated functions for each size of the Pareto front.

Table 1: Number of bi-objective functions for different sizes of the Pareto fronts

Size PF	1	2	3	4
Numb. functions	94295	190059	150841	66745
Size PF	5	6	7	8
Numb functions	17411	3990	375	60

For a given bi-objective function (g_a, g_b) , we compute first the Pareto set. Then, for each Pareto-optimal solution, we

compute the degree of deception in each objective. Finally, we define the degree of deception of the bi-objective function (g_a, g_b) , as the mean, computed among the objectives, of the average rank of all the solutions in the Pareto-set. This is a straightforward extension of the degree of deception defined for single-objective functions. Other ways to define the degree of deception for MOPs are possible (e.g. considering the minimum degree of deception among the Pareto-optimal solutions, quartile, mode, and other criteria.).

Figure 3 shows the histogram of the degree of deception for the bi-objective functions in $\mathcal{O}$.

We assume that, if the Pareto-optimal solutions are deceptive, then MOEAs will spend more computational effort to find these Pareto-optimal solutions or will miss them.

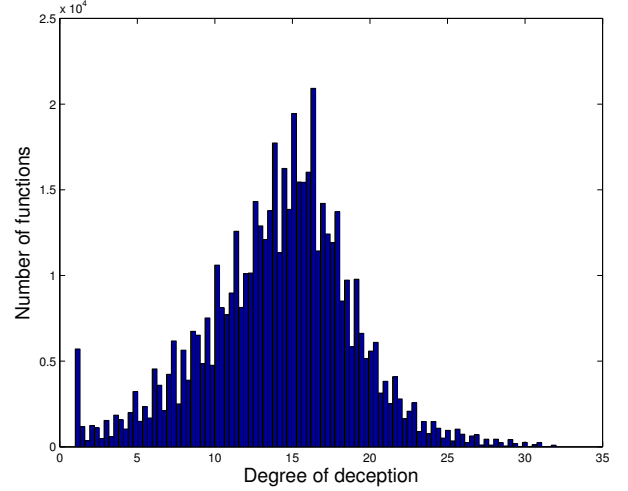


Figure 3: Histogram of the degree of deception for functions in $\mathcal{O}$.

5. DECEPTIVE MUBQP INSTANCES

In the previous section, we have explained how to create bi-objective deceptive functions and compute the degree of deception. We use $\mathcal{O}$ as a source of bi-objective functions (submatrices) with a different number of solutions in the Pareto set and different degrees of deception. We will use these submatrices to apply the planting procedure explained in Section 4 and create larger deceptive mUBQP instances.

We define a deceptive mUBQP problem using the parameters $\{n, M, d, l\}$ where n is the number of variables, M is the number of objectives, d is the density of zeros in all the M matrices that define the problem, and l is the amount of injected deception. We focus in the bi-objective case, i.e. $M = 2$.

Parameters $\{n, M, d\}$ are used in [12], where a generator of mUBQP instances is proposed. We define $l = \frac{k}{n'}$, where k is the number of deceptive UBQP functions ($n = 5$) injected to a traditional mUBQP instance generated with parameters $\{n', M, d, c\}$, where $n' \gg 5$. The steps to generate the mUBQP instances are shown in Algorithm 2. The algorithm receives as input a set of seeding functions $\mathcal{O}' \subset \mathcal{O}$ explained above.

We designed five different datasets containing instances created using different criteria. The first three datasets in-

Algorithm 2: Generation of large mUBQP instances

```

1  Initialize matrices  $Q_1$  and  $Q_2$  to zero
2  For  $i = 1$  to  $n/l$ 
3    Select a random set of 5 variables
4    Select a random function  $(g_a, g_b)$  from the seed-
      ing set of bi-objective functions
5    Substitute cells corresponding to the selected
      variables in  $Q_1$  by  $g_a$ , and in  $Q_2$  by  $g_b$ 

```

clude instances generated using $\mathcal{O}$. The first step consists in selecting a set of seeding submatrices from $\mathcal{O}$. Each type of instance is defined by the way this set is selected. Then, a common algorithm is invoked to generate the instances from the set. All instances used in this paper have dimension $n = 100$.

- (T1) *HardInst*: Comprises problems with at least 7 non-dominated solutions and deceptive values above 24. 10 instances.
- (T3) *LessPopInst*: The set comprises the bi-objective problems in $\mathcal{O}$ that have at least 4 non-dominated solutions and the degree of deception is above 29. 9 instances.
- (T4) *ComposeInstHard*: Comprises problems with at least 7 non-dominated solutions. The 100 such problems in $\mathcal{O}$ with maximum degree of deception are selected. 10 instances.

The second dataset has been manually engineered to include typical cases of deception. The last dataset contains random instances generated according to the method proposed in [11].

- (T2) *ArtTypeInst*: Each of the instances corresponds to a different problem manually defined. 5 instances.
- (T5,T6,T7) *RandomInst*: The sparsity of the matrices was respectively set to 0.8, 0.02, and 0.03. For each instance type, one instance was generated for each correlation value in $\{-0.5, 0, 0.5\}$. Therefore there were 3 instances for each type.

6. MOEA/D: A DECOMPOSITION-BASED APPROACH TO MUBQP PROBLEM

MOEA/D algorithms [27] decompose a MOP into a number (N) of scalar single objective optimization subproblems and optimize them simultaneously in a collaborative manner using the concept of neighborhood between the subproblems. Each subproblem i is associated to a weight vector $\lambda^i = (\lambda_1^i, \dots, \lambda_m^i)^T$ and the set of all weight vectors is $\{\lambda^1, \dots, \lambda^N\}$. We use the *Tchebycheff* approach as the scalar aggregation function. The optimal solution to the following scalar single-optimization problems is defined as:

$$\begin{aligned} \text{minimize } g^{tc}(\mathbf{x}|\lambda, z^*) &= \max_{1 \leq l \leq m} \{\lambda_l | f_l(\mathbf{x}) - z_l^*|\}; \\ &\text{subject to } \mathbf{x} \in \Omega \end{aligned} \quad (7)$$

where $z^* = (z_1^*, \dots, z_m^*)^T$ is the reference point, i.e., $z_l^* = \min\{f_l(\mathbf{x}) | \mathbf{x} \in \Omega\}$ ¹ for each $l = 1, \dots, m$. For each Pareto

¹maximization problem: $z_l^* = \max\{f_l(\mathbf{x}) | \mathbf{x} \in \Omega\}$

optimal solution x^* there exists a weight vector λ such that x^* is the optimal solution of (7) and each optimal solution of (7) is Pareto optimal of (1).

MOEA/D-GM [4] is a general framework comprising different classes of EAs. The emphasis of the framework is how to integrate efficient ways to exploit the information contained in the neighborhoods. Therefore, probabilistic modeling methods are included in the framework, although it can also accommodate traditional crossover operators.

In addition to a simple GA, we consider in this paper two EDAs that use univariate probabilistic models: the univariate marginal distribution algorithm (UMDA) [16] and population-based incremental learning (PBIL) [2]. Our goal is not to focus on the comparison between the algorithms but evaluate the difficulty of the instances introduced in the paper in a wider context, considering EAs with different search strategies. UMDA and PBIL are similar, but the adaptation of the univariate model in PBIL is more parsimonious. The probabilistic model in PBIL is slightly varied at each generation based on a mutation rate parameter. For more details on UMDA and PBIL [2, 13, 16] can be consulted. In Algorithm 3 the steps of the MOEA/D-GM framework are shown.

Algorithm 3: MOEA/D-GM

```

1  [ $Pop, B(i), R(i), EP$ ]  $\leftarrow$  Initialization() //  $Pop = \{1, \dots, N\}$  is
    the initial population of size  $N$ .  $B(i)$  and  $R(i)$  are, respec-
    tively, the neighbors for selection and replacement for each
    subproblem  $i$ .  $EP$  is the external population.
2  While a termination condition not met
3    For each subproblem  $i \in 1, \dots, N$  at each generation
4      Learning()
5        Case GA: Choose two parent solutions  $x^1 x^2$  from  $B(i)$ 
6        Case UMDA: Learn a probabilistic vector  $p^i$  using the
          solutions from  $B(i)$ 
7        Case PBIL: Learn an incremental probabilistic vector
           $p^i$  using the solutions from  $B(i)$ 
8      Sampling()
9        Case GA: Apply the crossover and mutation to gener-
          ate  $\mathbf{y}$ 
10       Case PBIL or UMDA: Sample  $\mathbf{y}$  using the probability
          vector  $p^i$ 
11       Compute the fitness function  $F(\mathbf{y})$ 
12       Update_Population( $Pop, \mathbf{y}, R(i)$ )
13       Update_EP( $F(\mathbf{y}), EP$ )
14  Return  $Pop, EP$ 

```

7. EXPERIMENTS

In addition to the exploratory analysis to investigate the space of mUBQP deceptive functions presented in Section 4, we empirically evaluate in this section the way in which the characteristics of the generated instances influences the behavior of MOEAs. We apply the variants of MOEA/D-GM explained in the previous section. First we introduce the performance metrics used to compare the behavior of the algorithms and explain the criteria considered to evaluate the influence of the instances. Then we present and discuss the numerical results of the algorithms.

7.1 Performance metrics

To asses the quality of the approximated Pareto fronts obtained by each algorithm, a well-known quality indicator

is used: The modification of the Inverted Generational Distance (IGD) known as IGD_p [19, 23]. This indicator, as the authors demonstrated, is more Pareto compliant than the original IGD.

Let P^* be a set of uniformly distributed Pareto optimal solutions along the *true PF* in the objectives, and P be an approximated set to the *true PF* obtained by an algorithm.

IGD_p metric [28]: The inverted generational distance from P^* to P is defined as

$$IGD_p(P^*, P) = \frac{\sum_{v \in P^*} d(v, P)}{|P^*|} \quad (8)$$

where $d(v, P)$ is the minimum Euclidean distance between v and the solutions in P . If $|P^*|$ is large enough $IGD_p(P^*, P)$ could measure both convergence and diversity of P in a sense. The lower the $IGD_p(P^*, P)$, the better the approximation of the obtained P to the true PF.

We will measure the difficulty of the instances in terms of the ability of the algorithms to optimize this metric and the number of generations needed to produce a low IGD_p . Therefore, as a preliminary step and in order to compute the true Pareto, we have exhaustively executed the algorithms to produce a reference Pareto front (P^*) for each instance. The algorithms were guaranteed a large number of function evaluations (5000 generations) to find accurate PF approximations. In a second step of the experiments, we have used the produced reference fronts to compute the IGD_p metric and size of the PF in each generation of the algorithms. From this information we could determine whether the IGD_p metric was minimized at the end of the search and the number of generations needed for minimizing it.

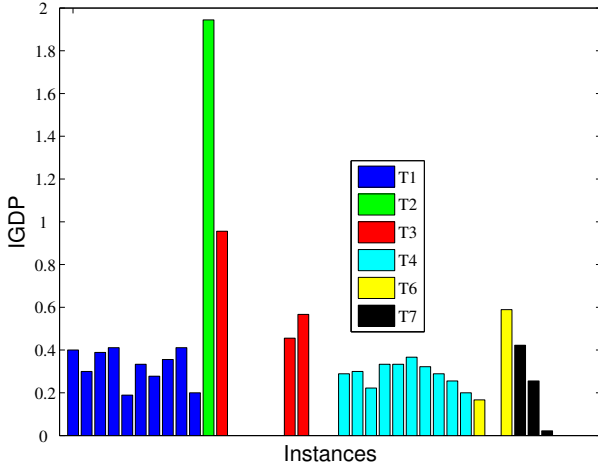


Figure 4: Average IGD_p value achieved in the last generation of the algorithms for all instances.

7.2 Parameters setting

The following parameters settings were used in the experimental study:

1. *Number of subproblems (N):* The number of subproblems N and their correspondent weight vectors $\lambda^1, \dots, \lambda^N$ are controlled by a parameter H , which makes a wide spread distribution of the weight vectors according to

$N = C_{H+m-1}^{m-1}$. Thus, for the bi-objective problem, we set $H = 100$, consequently $N = 101$.

2. *Selection and replacement neighborhood size:* The number of the selected solution S for each $B(i)$ and $R(i)$ is set to $0.2 \times N$, i.e., $S = 20$.
3. *Maximal number of replacements by a new solution:* Each new generated solution y can update a maximum $n_r = 2$ parents solution from $R(i)$.
4. *Genetic operators from MOEA/D-GA:* uniform crossover and mutation rate $\mu = \frac{1}{n}$.
5. *PBIL learning rate:* $\alpha = 0.02$.
6. *Number of generations (second phase):* $g = 500$.

For each of the 43 instances included in our study, the three variants of MOEA/D were executed 30 times.

7.3 Analysis of the reference PFs

Table 2: Size of the reference Pareto front approximations computed for the test instances. Instances for which $q_{ij}^k \notin \{\pm 1\}$ are underlined.

index	T1	T2	T3	T4	T5	T6	T7
1	121	101	80	69	<u>500</u>	43	50
2	121	<u>4921</u>	80	72	<u>289</u>	22	23
3	121	<u>243</u>	80	67	<u>140</u>	13	11
4	121	<u>9157</u>	80	71			
5	121	<u>174</u>	80	70			
6	121		80	65			
7	141		60	67			
8	121		60	70			
9	121		60	59			
10	121			69			

Table 2 shows the size of the PFs for all the instances. One of the findings from the analysis of the PFs was the observation that the most important factor in the size of PFs is the range of values from which the mUBQP instances are generated. The higher the range of the interval from which values of q_{ij}^k are sampled the wider the range of the output values of the functions and the PF. This is an issue that is usually overlooked. The PF can be arbitrarily enlarged by increasing the range of values in the matrices, but this does not necessarily imply a more “interesting” or difficult PF. In Table 2, instances for which q_{ij}^k were selected for a range different than $\{\pm 1\}$ are underlined. In the following, we focus on functions for which $q_{ij}^k \in \{\pm 1\}$.

7.4 Influence of the instance difficulty in the behavior of the algorithms

We computed the average IGD_p achieved by the algorithms in the last generation and the average number of generations needed to achieve an optimal value of the IGD_p . When the optimal value of IGD_p is not reached in a run we assign a value higher than the number of generations. Figures 4 and 5 respectively show the average IGD_p and the average number of generations reached for all the instances. The values shown are an average of the results for all the algorithms, i.e. 90 runs.

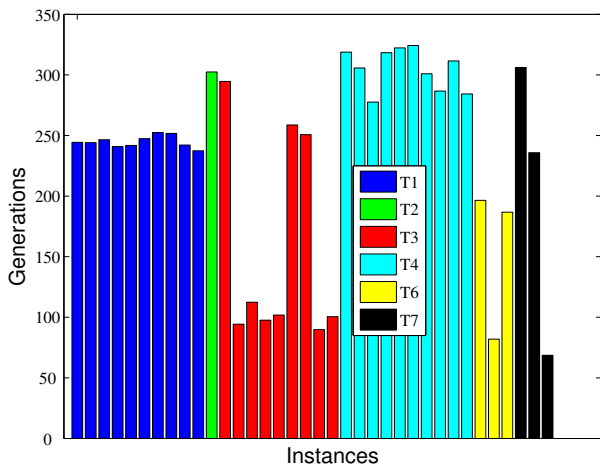


Figure 5: Average number of generations needed to reach an optimal IGD_p value for all instances.

An analysis of Figure 4 reveals that the instance with the highest IGD_p corresponds to one of the manually designed instances. In this case corresponds to the problem with negative correlation between the objectives -1 and PF size equal 101. After this instance, the most difficult instances in terms of the IGD_p belongs to the set *LessPopInst*. This is perhaps the set of instances with the highest variability since it also contains instances that can be easily solved (IGD_p equals 0) by all the algorithms. Random instances with sparsity 0.02 and correlation 0 and negative are easy to optimize.

Nevertheless, the analysis of the behavior of the algorithms can not be completed without evaluating the time spent by the variants of MOEA/D to reach good approximations of the PF. It can be seen in Figure 5 that the number of generations to find an optimal IGD_p measure changes considerably among the different classes of instances. All instances *ComposeInstHard* and *InstHard* required on average between 50 and 200 more generations than the random instances with the same sparsity (T6). As expected, increasing the number of non-zero values in the matrices (T7) allows having more populated PFs and the algorithms need more generations to obtain good approximations.

Another potential use of instances is to investigate the behavior of different algorithms. We compare the behavior of MOEA/D-GA, MOEA/D-PBIL, and MOEA/D-UMDA in all instances in classes $\{T_1, T_3, T_4, T_6, T_7\}$. Figure 6 shows the average distribution of the IGD_p values for all the algorithms. To compute the distribution, we have first created a set containing IGD_p values for all instances in each class. Then a violin plot is created for each class showing the distribution of the values. It can be seen in Figure 6 that the MOEA/D variants exhibit a clear different behavior for instances in class T3.

8. CONCLUSIONS AND FUTURE WORK

In this paper, we have proposed a new way to generate hard instances for the mUBQP problem. Our algorithm is based on planting deceptive subsolutions in the matrices that define the mUBQP problem. We have empirically investigated the difficulty of the instances using different vari-

ants of the MOEA/D algorithm that add probabilistic-based operators. Our experimental results confirm that the introduced instances are indeed harder for MOEA/D than instances that are randomly generated. Our study also throw light on a number of issues that influence the complexity of mUBQP instances and that should be taken into account at the time of using this problem as a MOEA testbed.

9. ACKNOWLEDGMENTS

This work has received support by IT-609-13 program (Basque Government) and TIN2013-41272P (Spanish Ministry of Science and Innovation). It has also received support by CNPq Productivity Grant Nos. 306103/2015-0, CNPq Program Science Without Borders Nos.: 400125/2014-5, and CAPES program 99999.010574/2014-00, all financed by the Brazil Government.

10. REFERENCES

- [1] H. E. Aguirre and K. Tanaka. Working principles, behavior, and performance of MOEAs on MNK-landscapes. *European Journal of Operational Research*, 181(3):1670–1690, 2007.
- [2] S. Baluja. Population-based incremental learning: A method for integrating genetic search based function optimization and competitive learning. Technical Report CMU-CS-94-163, Carnegie Mellon University, Pittsburgh, PA, 1994.
- [3] C. C. Coello, G. Lamont, and D. van Veldhuizen. *Evolutionary Algorithms for Solving Multi-Objective Problems*. Genetic and Evolutionary Computation. Springer, Berlin, Heidelberg, 2nd edition, 2007.
- [4] M. Z. de Souza, R. Santana, A. T. R. Pozo, and A. Mendiburu. MOEA/D-GM: using probabilistic graphical models in MOEA/D for solving combinatorial optimization problems. *CoRR*, abs/1511.05625, 2015.
- [5] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler. Scalable test problems for evolutionary multiobjective optimization. In *Evolutionary Multiobjective Optimization*, Advanced Information and Knowledge Processing, pages 105–145. Springer London, 2005.
- [6] B. Derbel, D. Brockhoff, A. Liefvooghe, and S. Verel. On the impact of multiobjective scalarizing functions. In *Parallel Problem Solving from Nature-PPSN XIII*, pages 548–558. Springer, 2014.
- [7] D. E. Goldberg, K. Deb, and J. rey Horn. Massive multimodality, deception, and genetic algorithms. IlliGAL Report 92005, University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory, Urbana, IL, 1992.
- [8] S. Huband, P. Hingston, L. Barone, and L. While. A review of multiobjective test problems and a scalable test problem toolkit. *Evolutionary Computation, IEEE Transactions on*, 10(5):477–506, 2006.
- [9] J. Knowles and D. Corne. Instance generators and test suites for the multiobjective quadratic assignment problem. In *Evolutionary Multi-criterion Optimization*, pages 295–310. Springer, 2003.
- [10] G. Kochenberger, J.-K. Hao, F. Glover, M. Lewis, Z. Lü, H. Wang, and Y. Wang. The unconstrained binary quadratic programming problem: a survey.

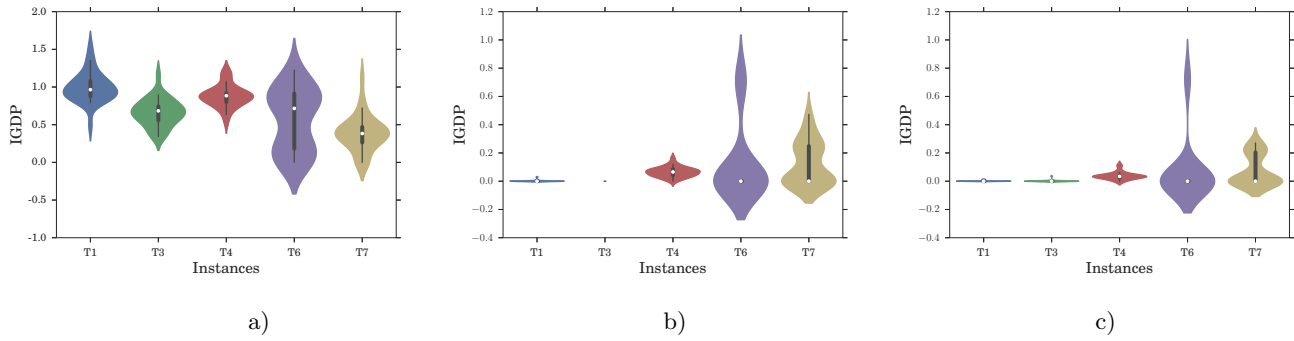


Figure 6: Distribution of the IGD_p metric achieved by MOEA/D variants for instances in classes $\{T_1, T_3, T_4, T_6, T_7\}$. a) GA; b) PBIL; c) UMDA.

- Journal of Combinatorial Optimization*, 28(1):58–81, 2014.
- [11] A. Liefooghe, S. Verel, and J.-K. Hao. A hybrid metaheuristic for multiobjective unconstrained binary quadratic programming. *Applications Soft Computing*, 16:10–19, 2014.
 - [12] A. Liefooghe, S. Verel, L. Paquete, and J.-K. Hao. Experiments on local search for bi-objective unconstrained binary quadratic programming. In *Evolutionary Multi-Criterion Optimization*, pages 171–186. Springer, 2015.
 - [13] L. Lozada-Chang and R. Santana. Univariate marginal distribution algorithm dynamics for a class of parametric functions with unitation constraints. *Information Sciences*, 181(11):2340–2355, 2011.
 - [14] G. Marquet, B. Derbel, A. Liefooghe, and T. El-Ghazali. Shake them all! rethinking selection and replacement in MOEA/D. In *Parallel Problem Solving from Nature - PPSN XIII International Conference*, volume 8672, pages 641–651, 2014.
 - [15] J. P. Martins, A. H. M. Soares, D. V. Vargas, and A. C. B. Delbem. Multi-objective phylogenetic algorithm: Solving multi-objective decomposable deceptive problems. In *Evolutionary Multi-Criterion Optimization*, pages 285–297. Springer, 2011.
 - [16] H. Mühlenbein. The equation for response to selection and its use for prediction. *Evolutionary Computation*, 5(3):303–346, 1997.
 - [17] H. Mühlenbein, T. Mahnig, and A. Ochoa. Schemata, distributions and graphical models in evolutionary optimization. *Journal of Heuristics*, 5(2):213–247, 1999.
 - [18] M. Pelikan, K. Sastry, and D. E. Goldberg. Multiobjective hBOA, clustering and scalability. IlliGAL Report No. 2005005, University of Illinois at Urbana-Champaign, Illinois Genetic Algorithms Laboratory, Urbana, IL, February 2005.
 - [19] C. A. Rodríguez Villalobos and C. A. Coello Coello. A new multi-objective evolutionary algorithm based on a performance assessment indicator. In *Proceedings of the 14th annual conference on Genetic and evolutionary computation*, pages 505–512. ACM, 2012.
 - [20] R. Santana, A. Mendiburu, and J. A. Lozano. Computing factorized approximations of Pareto-fronts using mNM-landscapes and Boltzmann distributions. *CoRR*, abs/1512.03466, 2015.
 - [21] R. Santana, A. Mendiburu, and J. A. Lozano. Evolving MNK-landscapes with structural constraints. In *Proceedings of the IEEE Congress on Evolutionary Computation CEC 2015*, pages 1364–1371, Sendai, Japan, 2015. IEEE press.
 - [22] R. Santana, A. Ochoa, and M. R. Soto. Factorized Distribution Algorithms for functions with unitation constraints. In *Evolutionary Computation and Probabilistic Graphical Models. Proceedings of the Third Symposium on Adaptive Systems*, pages 158–165, 2001.
 - [23] O. Schütze, X. Esquivel, A. Lara, and C. A. C. Coello. Using the averaged hausdorff distance as a performance measure in evolutionary multiobjective optimization. *Evolutionary Computation, IEEE Transactions on*, 16(4):504–522, 2012.
 - [24] S. Verel, A. Liefooghe, L. Jourdan, and C. Dhaenens. Pareto local optima of multiobjective NK-landscapes with correlated objectives. In *Evolutionary Computation in Combinatorial Optimization*, pages 226–237. Springer, 2011.
 - [25] S. Verel, A. Liefooghe, L. Jourdan, and C. Dhaenens. On the structure of multiobjective combinatorial search space: MNK-landscapes with correlated objectives. *European Journal of Operational Research*, 227(2):331–342, 2013.
 - [26] D. Whitley. Mk landscapes, NK landscapes, MAX-kSAT: A proof that the only challenging problems are deceptive. In *Proceedings of the 2015 on Genetic and Evolutionary Computation Conference*, pages 927–934. ACM, 2015.
 - [27] Q. Zhang and H. Li. MOEA/D: A multiobjective evolutionary algorithm based on decomposition. *IEEE Transactions on Evolutionary Computation*, 11(6):712–731, 2007.
 - [28] E. Zitzler and L. Thiele. Multiobjective evolutionary algorithms: A comparative case study and the strength pareto approach. *Evolutionary Computation, IEEE Transactions on*, 3(4):257–271, 1999.